MATH 371 Final Project

University of Pennsylvania

Department of Mathematics

Category Theory and Computer Science

Authors:
Gregory Zhu
Angelica Sinay

Professor:
Maximilien Peroux

October 28, 2026

Contents

1 Introduction **3**
- 1.1 Background Needed . . . 3
- 1.2 Haskell . . . 3
  - 1.2.1 Language Syntax . . . 3

2 Functors **5**

3 Monoids **7**
- 3.1 Definition, with and without category theory . . . 7
- 3.2 Monoids in Haskell . . . 7
- 3.3 Examples . . . 7

4 Monads **10**
- 4.1 Monads, categorically speaking . . . 10
  - 4.1.1 Definition . . . 10
  - 4.1.2 Examples . . . 10
  - 4.1.3 Kleisli Category . . . 13
- 4.2 Monads from a Programmer's Point of View . . . 13
  - 4.2.1 Haskell Monad Definition . . . 14
  - 4.2.2 Using Monads in Haskell . . . 15
  - 4.2.3 Summary of Monads . . . 18

5 Conclusion **19**

A First Appendix **20**

1. Introduction

In computer science, there are different paradigms of computation. Most popular is object oriented programming (OOP). This paradigm is central in many of the worlds most popular languages such as Java and C++. Object oriented programming is focused around objects. Objects have states and methods which manipulate the internal states. However, another paradigm is functional programming. As its namesake implies, the central focus is the functions themselves. In functional programming, functions are first-class. Functions are composable, they can be passed as arguments into other functions, and in general, they have no second order effects (we'll talk about this later). Its this paradigm of programming that aligns very closely with category theory. Category theory is all about objects and morphisms (arrows). By modeling functions as morphisms in a category, composition takes the format. Category theory offers rich concepts to study the composition of functions. Using category theory concepts, we can better leverage the power of functional programming.

1.1 Background Needed

In general, basic understanding of category theory is assumed. As this is a MATH371 final project, we do not assume any computer science background or prior experience with Haskell. Because of this, we will do our best to explain Haskell code and syntax and translate those into more familiar notation when necessary.

1.2 Haskell

Haskell is a statically typed, purely functional programming language. What does this mean? Statically typed means that once a variable is declared, the type cannot be changed. In general, this adds a level of safety when programming. If you know the type cannot be changed you can program with that assumption. Actually, most variables are immutable in Haskell. This means that once a value is assigned to a variable, that can never be changed. This brings us to the second descriptor "purely functional". This means that in Haskell, functions are "pure". That is, they don't mutate data. Because of this, in Haskell, functions have predictable inputs and outputs. Given an input, the output will always be the same. This is not a general assumption you can make in programming. This purity allows us to make powerful guarantees about the composition of functions that will allows us to apply category theory to the language.

1.2.1 Language Syntax

As in all programming languages, typing plays a major role in the language. Types are simply sets of objects. For example the type **Int** in Haskell is simply the set of all integers in $\mathbb{Z}$.

```
1  -- Declaring Type in Haskell
2  b :: Int
3  -- Assigning Value
4  b = 23
5  -- This would be invalid assignment since we have declared the type of b to be Int
6  b = "Hello"
```

Another central part of Haskell are functions. To declare a function:

```
1  -- Function Type Definition
2  sum :: Int -> Int -> Int
3  -- Function declaration
4  sum a b = a + b
```

Notice the weird notation **Int -> Int -> Int**. In normal mathematical notation, we are declaring some function sum : $\mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$. The last type in a Haskell type declaration is the output of a function and the rest are inputs. Actually, we're cheating a bit since + is already defined in Haskell as a function `(+) :: Int -> Int -> Int` which is normal integer addition. The parentheses

around the + operator mean that it can be used in between the two parameters to the function. So, in Haskell we can write `3 + 4` which is equivalent to **sum** `3 4`.

We can also declare our own types as well. Using the **data** keyword we can declare new datatypes such as,

```
-- type keyword introduces a synonym to an already existing type
type Center = (Float, Float)
type Radius = Float
-- declare a new data type
data Circle = Circle Center Radius
```

We now can define a function using our new type.

```
-- Area of circle function
area :: Circle -> Float
area (Circle _ r) = pi * r ^ 2
```

As mentioned before, functions are first class in Haskell. This means we can treat them as data types and pass them as argument to functions. Consider a list $[1, 2, 3, 4, 5]$. Say we want a function that adds all the elements in the list together. With functional programming, we can generalize this notion. Instead of writing a specific function to do this, we can instead create a function that "collapses" this list together. In the example earlier this was `(+)`. In Haskell we call this the `fold`.

```
-- The fold function definition (notice that it's recursive!)
fold :: (a -> b -> b) -> b -> [a] -> b
fold f z [] = z
fold f z (x:xs) = f z (fold f z xs)

-- We can now add all the numbers in our list
fold (+) 0 [1,2,3,4,5] = 15

-- We can also multiply
fold (*) 1 [1,2,3,4,5] = 120

-- We now can create a factorial function using fold!
fact n = fold (*) 1 [1..n]
```

Currying

Another really important part of functional programming is the ability to curry functions. What the heck is currying? Currying is simply getting a new function by filling in some but not all the arguments of a function. In category theory, this would be,

$$\mathrm{Hom}_{Set}(X \times Y, Z) \cong \mathrm{Hom}_{Set}(X, \mathrm{Hom}_{Set}(Y, Z))$$

This is adjunction! We have a function $f : X \times Y \to Z$ and we can create a new function $f' : Y \to Z$ by passing in some $x \in X$. In Haskell the syntax is simply:

```
-- The sum function from earlier
sum :: Int -> Int -> Int
sum a b = a + b

-- currying the function
plusTwo :: Int -> Int
plusTwo = f 2
```

From our sum function, we now get a new function that takes only one argument and adds two to that argument. That's currying! We'll be seeing a lot more of this for sure.

2. Functors

Ok, let's talk about functors in Haskell. We're assuming prior knowledge on functors in the categorical sense. As a recap, we have a functor $F : \mathcal{C} \to \mathcal{D}$. What we want to do is to create a map between different categories. But, categories have two different type of data, we have objects and morphisms. So, what a functor really is is two different maps.

$$F : \mathcal{C} \to \mathcal{D}$$
$$C \mapsto D$$
$$(C \xrightarrow{f} C') \mapsto (F(C) \xrightarrow{g} F(C'))$$

So, what does this look like in Haskell? Actually there is an extremely similar notion of this concept in Haskell. In Haskell, a functor is defined like this,

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
```

Ok, let's unpack what this Haskell definition is saying. Here, we have this definition for **Functor** `f` which takes in some type `f`. For that type we must supply some `fmap` operation. Since the type is `fmap :: (a -> b) -> f a -> f b` that means fmap takes some function `a -> b` and then supplies another function `f a -> f b`. Notice that this is exactly like the morphism map with categorical functors. We are taking in some function (a morphism) and we are supplying a new function in our new typeclass `f`.

Ok, so it's not perfectly one to one. We're not really moving to a different category. What we're really doing is moving from our category `Hask` to the subset of `Hask` that is only the types of `f`. So really, a functor is an endofunctor in the category `Hask` that takes morphisms in `Hask` to a subset. It's a bit weird, but we'll see in some examples why this computationally makes sense.

Example: Lst

So, let's consider the functor `Lst`. `Lst` is the parametric type list, which is simply a list of some types. For example we can have `Lst` **Int** which is simply a list of ints. In general, we denote this as `[Int]`. As a functor, this is,

```
instance Functor Lst where
    fmap _ []    = []
    fmap f (x:xs) = f x : fmap f xs
```

Here, we're saying that if we have any function and the empty list, we simply get the empty list. Otherwise, for each item in the list, we apply a function `f` to it. In functional programming, this is usually just known as **map**. For example, if we have,

```
fmap (+1) [1,2,3,4,5] = [2,3,4,5,6]
```

We can apply the function `(+1)` which increments an Int by 1 to the list `[1,2,3,4,5]` which applies `(+1)` to each element in the list. As we said earlier, `fmap` is a way for us to translate functions from `Hask` into functions in `Lst` which is just the category of list types in Haskell.

Example: Maybe

So, let's consider another functor which is the **Maybe** parametric type. **Maybe** is a way to encode this idea of failed computation. For example, let's define some division function **div** `::` **Float** `->` **Float** `->` **Float**. So, **div** takes in two arguments a and b and returns $\frac{a}{b}$. But what if $b = 0$. Then we have problems.

Normally, in programming we solve this by having what's called an exception. This means that the computation fails for some reason. Usually this is for (pun intended) exceptional cases. However, what if dividing by 0 is a normal occurrence in our program and we want to process that and move on? That's when **Maybe** really shines. The **Maybe** class is defined as,

```
data Maybe a = Just a | Nothing}
```

Just `a` is simply something of type `a`. Consider it like a container or box around the object. Then we also have **Nothing** which is what it sounds like, nothing!

So if we redefine our **div** function from earlier as **div** `::` **Float** `->` **Float** `->` **Maybe Float**, we can encode this idea of failed computation. If **div** `a b` is valid then we return $c = \frac{a}{b}$ as **Just** `c`. If `b == 0`, then we return **Nothing**. Here's the full function definition,

```
div :: Float -> Float -> Maybe Float
div _ 0 = Nothing
div a b = a / b
```

Now, let's talk about **Maybe** as functor. To do this, we only need to define this notion of `fmap`.

```
instance Functor Maybe where
    fmap _ Nothing = Nothing
    fmap f Just x = Just (f x)
```

So, this is a way for us to translate type **Maybe** `a` to type **Maybe** `b`. If it's **Nothing**, then we don't do anything! But otherwise we apply our function `f` to the item `x`. As you can see, `fmap` is a way to translate normal functions into functions over our "new" category which would be the category of **Maybe**'s. We'll be seeing a lot more of the **Maybe** class in our chapter about monads.

Functor Laws

What about the functor laws? In category theory we have two functor laws. That is $F(id_C) = id_{(F(C))}$ and $F(f \circ g) = F(f) \circ F(g)$. Now what about in Haskell? Well, it turns out that there is the same notion of composability. However, unfortunately this is not enforced by the Haskell compiler. So, it's up to the programmer to ensure that a Functor they define is consistent with these laws. All the standard library functors in Haskell follow this rule. It's also not too hard to check. So, in Haskell this would be,

```
-- Functor Laws
fmap id = id
fmap (f . g) = fmap f . fmap g
```

If we were to check with **Maybe** and `Lst`, we would be able to see that this is indeed true with their implementations.

3. Monoids

3.1 Definition, with and without category theory

First up, monoids. Recall, in class we learned that monoids are special types of constructions with three main characteristics. Without delving into any category theory, we are essentially dealing with a set of objects, say M with a binary operation $*$ such that

- M is closed under $*$
- $*$ is unital, i.e. there exists an element $e \in M$ such that $\forall m \in M,\ e * m = m = m * e$
- $*$ is associative, that is for $x, y, z \in M,\ x * (y * z) = (x * y) * z$

Almost exactly a group, but without inverses!

In the world of category theory, we recognize that monoids are exactly categories with one object, call BM where $\text{Ob}(BM) = \{\star\}$ any fixed one point set. $Hom_{BM}(\star, \star) = M$ the elements of set M, and

$$\begin{aligned} Hom_{BM}(\star,\star) \times Hom_{BM}(\star,\star) &\to Hom_{BM}(\star,\star) \\ (x, y) &\mapsto x * y \end{aligned}$$

3.2 Monoids in Haskell

In Haskell, monoids are already a predefined type class that looks like this:

```
1  class Monoid m where
2      mempty :: m
3      mappend :: m -> m -> m
4      mconcat :: [m]-> m
5      mconcat = foldr mappend mempty
```

Here,

1. Our binary operator $*$ is `mappend` that takes two monoid values and returns a third monoid value. Note that even though it is labeled "append", this does not necessarily mean this function *appends* something. Right now, all we know is that it is a function that begins with two values of one type and returns some value of the same type.

   Here, `x mappend mempty = x = mempty mappend x`.

2. Our identity element is `mempty` that does not really do anything but return the same value, similar to that of e in our monoid definition above.

   Similarly, `(x mappend y) mappend z = x mappend (y mappend z)` is associative in Haskell.

3. The last function `mconcat` is more of a bonus here as `mempty` and `mappend` usually suffice to instantiate a monoid class. `mconcat` here is more of an efficiency tool as it does the same thing as `mappend` with the capability of taking in a *list* of monoid values and reducing it to a single value (calling `mappend` on the elements of the list). This function is interesting as by itself it is also a monoid function.

In Haskell though, our rules of associativity and identity aren't automatically implemented and it is up to the programmer to make these law through code.

3.3 Examples

Example: Lists

```
instance Monoid [a] where
    mempty = []
    mappend = (++)
```

Here, `mempty = []` is the identity, an empty string, and `mappend = (++)` is the list concatenation operator taking two lists and combining them into one list. Lists are monoids in this case as any two input of lists returns an output of something that is still of the type list, and this works associatively.

```
("neverneedy" mappend "everlovely" ) mappend "jewel"
"neverneedyeverlovelyjewel"
"neverneedy" mappend ("everlovely" mappend "jewel")
"neverneedyeverlovelyjewel"
"whoseshinereflectsonyou" mappend mempty
"whoseshinereflectsonyou"
```

Example: Product and Sum

In class, we've identified that nonnegatives integers along with the operator + addition or $*$ are monoids as well. These monoid laws hold with Haskell too. In addition the identity is 0, and multiplication it is 1, exactly the same outside the world of code.

```
0 + 22
22
(1 + 2) + 3
6
1 + (2 + 3)
6
```

Another way of implementing sums and products as monoids though instantiating these concepts using the monoid class:

```
instance Num a => Monoid (Product a) where
    mempty = Product 1
    Product x mappend Product y = Product (x * y)
```

where you can call it as one would a monoid instance

```
getProduct $ Product 2 mappend 11
22
getProduct $ Product 22 mappend mempty
22
```

Why someone might prefer calling `2 mappend 11` instead of `2 * 11`, I'm not too sure myself. But this is definitely a start in understanding how to use monoid instances in Haskell.

Now, *maybe* we should try a more convincing example.

Example: Maybe

```
instance Monoid a => Monoid (Maybe a) where
    mempty = Nothing
    Nothing mappend m = m
    m mappend Nothing = m
    Just m1 mappend Just m2 = Just (m1 mappend m2)
```

Everything looks familiar, but not too sure what's going on – let's unwrap this.

In the first line, we instantiate a Monoid of type "a". Here, "a" doesn't necessarily mean any specific type (such as int "Int" or booleans "bool"), "a" is just a general type we're committing to and not changing.

Now the line `mempty = Nothing` is our identity element, that is implemented in the next two lines to follow the unital axiom we need for our monoid ($e * m = m = m * e$).

Now the binary operator `mappend` takes in two elements, if either one is the identity, it returns exactly the non-identity element. If neither are the identity, then the contents of the two **Just** types get appended into the same **Just** type.

```
Nothing mappend Just "Taylor"
Just "Taylor"
Just (Product 2) mappend Just (Product 11)
Just (Product {getProduct = 22})
```

Now, the product monoid instance makes more sense. We can call monoidal product instances with other monoids like the Maybe monoid. Maybe monoids are very important in Haskell as they relieve programmers of the stress of dealing with functions that may have failed (these results will just pass as **Nothing**) or when we're dealing with partial functions where you're not entirely sure where every single thing in your domain should land in its image, but you have to define it to go *somewhere* to keep your code working.

We are definitely seeing more of our **Maybe** friend later, but now let's talk about something that follows quite naturally – the *Monad.*

4. Monads

In the world of computer science, monads are often considered mysterious and difficult to grasp. In traditional programming languages like Java and C++, a programmer may never even hear about monads. There is no such monad typeclass in these languages. However, the monad is an extremely useful and flexible concept in functional programming. Because it's used in such disparate cases it can be hard to understand the underlying structure of monads from examples alone. Luckily, we have our categorical toolbelt.

4.1 Monads, categorically speaking

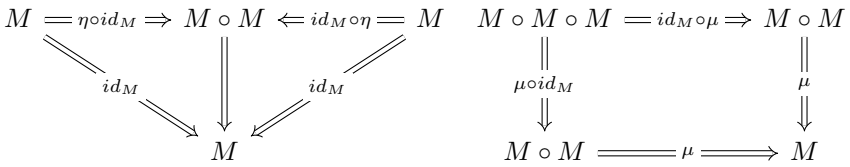
4.1.1 Definition

What are monads? Put simply, monads are monoids in the category of endofunctors. They're just a special case of our trustworthy monoids. We're already halfway there.

Let's go through our data for monoids, see how monads are different:

- In a monoid recall we have a set M, here our set M lives in the world of endofunctors, i.e. functors from a category to itself. Thus our $M \in Fun(\mathcal{C}, \mathcal{C})$ and its objects are functors from C to itself.
- Because we are dealing with functors, our binary operator will then be the natural transformation of our functors $\mu : M \circ M \Rightarrow M$
- Finally, our identity element e here will be the unit natural transformation $\eta : 1_C \Rightarrow M$

Now, we need all of these things such that we have the following diagrams commuting:



These diagrams seem quite abstract; let's ground ourselves in examples.

4.1.2 Examples

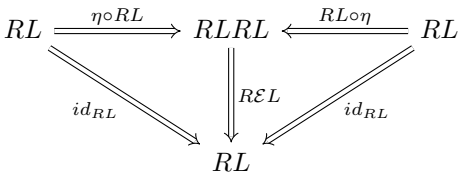
Example: Adjunction

Adjunction is such a good example of a monad, that there is even a lemma about it: *Any adjunction gives rise to a monad on the category C serving as the domain of the left adjoint, with:*

$C \underset{R}{\overset{L}{\rightleftarrows}} D$ where $L \dashv R$, unit $\eta : 1_C \Rightarrow RL$, counit $\mathcal{E} : LR \Rightarrow 1_C$

- the endofunctor $M := RL$ that goes from $C \to D \to C$
- the unit $\eta : 1_c \Rightarrow RL$ which is our $e : 1_C \Rightarrow M$
- the binary operator $RLRL \Rightarrow RL$ as the multiplication $M \circ M \Rightarrow M$

First, we know that this diagram commutes:



This is because in adjunction, we can use the counit $\mathcal{E} : LR \Rightarrow 1_C$ and see that

$$RLRL \Rightarrow R(LR)L \Rightarrow R(1_C)L \Rightarrow RL$$

Second, we know this diagram also commutes:

$$\begin{array}{ccc} RLRLRL & \xRightarrow{RLR\mathcal{E}L} & RLRL \\ {\scriptstyle R\mathcal{E}LRL}\Downarrow & & \Downarrow{\scriptstyle R\mathcal{E}L} \\ RLRL & \xRightarrow[R\mathcal{E}L]{} & RL \end{array}$$

Similarly, because we have the counit $\mathcal{E} : LR \Rightarrow 1_C$ we are able to get

$$RLRLRL \Rightarrow RLR(LR)L \Rightarrow RLR(1_C)L \Rightarrow RLRL$$

The argument for the other side of the square is almost identical.

Example: Maybe Monad

In category theory, the Maybe Monad is equivalent to the free $\dashv$ forgetful adjunction between ordinary and pointed sets.

$Set \underset{R}{\overset{L}{\rightleftarrows}} Set_*$ where $L \dashv R$, Set is the category of sets, and Set_* is the category of pointed sets.

Proof.

First, let's talk about what a pointed set is. In category Set_*,

- our objects are pairs (X, x_0) where $X \in Set$ and $x_0 \in X$ a fixed "pointer", hence the "pointed set"
- our morphisms are $f : (X, x_0) \to (Y, y_0)$ is any function $f : X \to Y$ such that $f(x_0) = y_0$

Now, we want to be able to define the data of this monad in such a way that the necessary diagrams commute.

For the first direction of the adjunction, we need:

$$\begin{aligned} L : Set &\to Set_* \\ X &\mapsto X_+ \end{aligned}$$

where $X \in Set$, $X_+ \in Set_*$. We are able to do this when, given $X \in Set$, we define a new pointed set $X_+ := X \bigsqcup *$, the disjoint union of X and the new point $*$, set to be pointer x_0.

And in the other direction (in its true forgetful fashion), we can define R as follows:

$$\begin{aligned} R : Set_* &\to Set \\ (X, x_0) &\mapsto X \end{aligned}$$

Defining the unit, we have

$$\begin{aligned} \eta : id_{Set} &\Rightarrow RL \\ \eta_X : X &\to X_+ \\ a &\mapsto a \end{aligned}$$

(we just map elements $a \in X$ to corresponding $a \in X_+$)

Now the counit,

$$\begin{aligned} \mathcal{E} : LR &\Rightarrow id_{Set_*} \\ \mathcal{E}_{(X,x_0)} : X_+ &\to (X, x_0) \\ x &\mapsto x \\ * &\mapsto x_0 \end{aligned}$$

(we just map each $x \in X$ to $x \in X_+$, then the extra point $*$ to x_0)

With all of these maps set-up, let us finally show this is a monad.

Now, our endofunctor $M := RL : (-)_+ : Set \Rightarrow Set$ (as it goes $Set \to Set_* \to Set$) where we map $X \mapsto X_+$ (going from $X \mapsto (X, x_0) \mapsto X_+$).

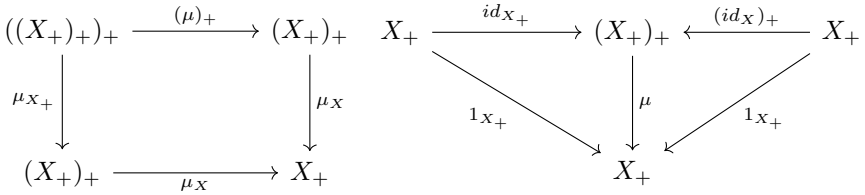
With this, we have the identity $id_{Set} \Rightarrow RL$ mapping $X \to X_+$.

Finally, we define our binary operator as follows

$$\begin{aligned} \mu : RLRL &\Rightarrow RL \\ (X_+)_+ &\to X_+ \\ x &\mapsto x \\ (+, +') &\mapsto + \end{aligned}$$

where we just map each $x \in (X_+)_+$ to $x \in X_+$ then map the two extra poiners $(+, +') \in (X_+)_+$ to pointer $+ \in X_+$.

As we've shown this to be a valid adjunction, we can use our adjunction lemma above to finish the proof showing this free $\dashv$ forgetful adjunction is a monad such that the diagrams commute.



Example: List Monad

Another example is the list monad, something a programmer might see as the concatenation of strings. In category theory, the list monad is also known as the **free monoid monad** that is another adjunction itself.

$Set \underset{R}{\overset{L}{\rightleftarrows}} Monoid$ where $L \dashv R$, Set is the category of finite sets, and $Monoids$ is the category of monoids. Here,

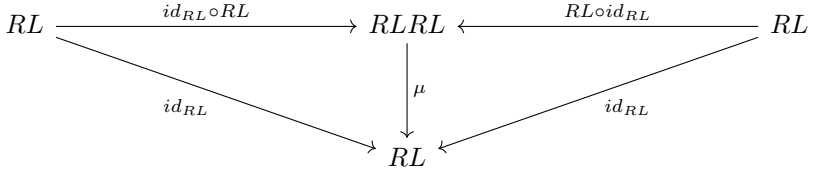
$$\begin{aligned} L : Set &\to Monoid \\ S &\mapsto \{\text{concatenated list of strings}\} \text{ for S = set of strings} \\ R : Monoid &\to Set \\ \{\text{concatenated list of strings}\} &\mapsto S \text{ (forgetting concatenation)} \end{aligned}$$

Now, the monad in this case comes with the following data:

- endofunctor $M := RL : Set \Rightarrow Set$ maps $S \mapsto \bigsqcup_{n \leq 0} S^n$ (disjoint union of strings of length n)
- $\mu : RLRL \Rightarrow RL$ sends {lists of lists} $\mapsto$ {lists on S}
- η : identity $id_{Set} \Rightarrow RL$ sends $S \mapsto [S]$ singleton list

{lists of singleton lists on S} $\longleftarrow\!\shortmid$ {lists on S}

{lists on S} $\longmapsto$ {singleton lists of lists on S}



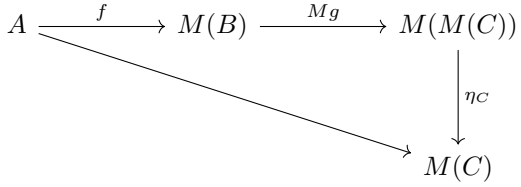
In other words,

$$\begin{aligned} RL \to & RLRL \\ S_1 \mapsto & [S_1] \\ & RLRL \leftarrow RL \\ & [[a_1], [a_2], ..., [a_n]] \leftarrow S_1 \ (a_i \in S_1) \end{aligned}$$

4.1.3 Kleisli Category

One interesting and very computer-science-relevant concept is the Kleisli category. And as with any category, we understand it by its data. If (M, μ, η) is a monad on C, a Kleisli category ($=: C_M$) is a subcategory of M such that:

- its objects are the objects of category C
- its morphisms are $\mu : A \to M(B) \in C$ when $A, B \in C$
- its identity $A \to A$ corresponds to $\eta_A : A \to M(A)$ from our monad
- and its composition of morphisms, $f : A \to M(B),\ g : B \to M(C)$ is defined as



Example: Kleisli on Maybe Monad

On our Maybe Monad, we have the Kleisli category with the data:

- our objects are sets in Set
- morphisms between $A \to B \in Set_{Maybe}$ are functions $f : A \to B_+$ where $A \in Set,\ B_+ \in Set_*$

which in computer science behaves like a "partially defined function on A" where most elements $a \in A$ are sent to some $b \in B_+$, but also allowing us to send some elements $x \in A$ to the pointer $+$ in B_+ possibly undefined.

Example: Kleisli on List Monad

On our List Monad, we have the Kleisli category with the data:

- our objects are sets of strings in Set
- morphisms between $A \to B \in Set_{Lists}$ are functions $f : S \to T$ where $S, T \in Set_{Lists}$

4.2 Monads from a Programmer's Point of View

As we've said earlier, category theory is all about morphisms and composability. Haskell too is all about morphisms and composability, albeit in the category **SET**. Monads play an important role in the story of composability. On top of just the usual composition of functions, monads allow you to extend composability even further whilst keeping your modularity and abstraction between functions.

Example: The Bank

Say you're a bank and you wish to create a functional computer system that manages the banks' transactions. For simplicity let's say that you have these functions:

```
-- These functions are very over-simplified to illustrate the point

-- login takes a string password and returns the account key
login :: String -> Int

-- withdraw takes an account key and return the amount withdrawn
withdraw :: Int -> Float

-- we can now compose them which takes some password and then withdraws from the account
withdraw . login
```

But now, a natural question, is what if we want to log our actions? In Haskell, functions are pure, which means that they have the same output for any input. Because of this, there are no global variables. This is a feature of Haskell. This is what facilitates composition. However, because of this, how would we create an audit trail for our bank program?

A natural solution is to pass in a log String and simply return the log string with the new log appended to it as an embellishment. Something like this,

```
-- old_login and old_withdraw would be the old implementations
-- of login and withdraw

-- Takes in two arguments and returns a pair
login :: String -> String -> (Int, String)
login x s = (old_login x, s') where
  s' = s ++ " login"

-- Takes in two arguments and returns a pair
withdraw :: Int -> String -> (Float, String)
withdraw x s = (old_withdraw x, s') where
    s' = s ++ " withdraw"
```

Now, this approach will work (with some issues that we won't discuss), but there are two issues now. One, we are breaking privacy since each function can see the entire banking log history. But an even bigger concern, now our functions don't compose! Let's solve this problem with monads.

4.2.1 Haskell Monad Definition

Monads in Haskell can be seen as extending the notion of composition that is so loved in functional programming. Let's take a look at the definition:

```
class Functor m => Monad m where
  return :: a -> m a
  (>>=) :: m a -> (a -> m b) -> m b
```

A Monad is defined with two operations, **return** and `(>>=)` which we'll refer to as bind. Like in category theory, monads are endofunctors. In this case, since every functor is an endofunctor in Haskell, that doesn't say much unfortunately.

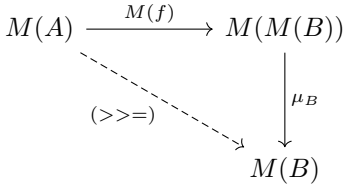
Exercise: Category Theory to Haskell

But, how does this relate to categorical monads? Well, we have two operations, just like in category theory. In category theory, we have two natural transformations,

$$\mu : M^2 \Rightarrow M$$
$$\eta : 1_C \Rightarrow M$$

We can see that η and **return**. Return is how we get our monad types from our regular types. Remember with Haskell functors? We didn't actually mention a way to transform objects into **Functor** types.

What about bind? We can actually construct this using μ. So, what is bind? Bind is a natural transformation $\mathbf{bind}_f : M \Rightarrow M$ together with some morphism $f : \mathcal{C} \to M(\mathcal{C})$, such that this diagram commutes:



So, given an f we are simply composing μ and $M(f)$.

What about the other direction? If we have bind defined, how do we get μ? Well, actually bind contains all the information we need. Consider $\mathbf{bind}_{id} : M \Rightarrow M$. If we input any $M(M(A))$ into bind, we get an output of $M(A)$. It's actually a bit easier to see with Haskell type definitions:

```
-- Normal type definition
(>>=) :: M a -> (a -> M b) -> M b

-- (>>=) with identity and input type a = (M x)
(>>=) :: M (M x) -> (M x -> M x) -> M x

-- another way to write eta (which is called join in Haskell)
join = x >>= id

-- for fun, we can define (>>=) via join which is the same as the commutative diagram
m >> f = join (fmap f m)
```

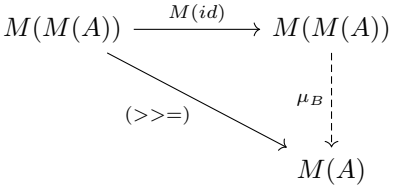
By substitution `M x` for type `a` and the identity for the morphism f, we can recover μ.

Let's take a look at it, using mathematical notation. We can view **bind** as a function:

$$\mathbf{bind} : M \times (\mathcal{C} \xrightarrow{f} M) \to M$$

where $M = M(\mathcal{C})$ is the monad. Then, you can see that if the input to bind is of the form $(M(M(X), id)$, then we recover an μ

As a commutative diagram,



Notice that $M(id)$ here is isomorphic, so that the diagram commutes. So, **bind** is isomorphic to μ for all $f : \mathcal{C} \to M$

4.2.2 Using Monads in Haskell

As we discussed earlier in the section about monoidal laws with monads. Monads must satisfy certain axioms for coherence. In Haskell, these must also be true. However, this is not checked for by the compiler so it is up to the programmer to ensure that these hold. These are,

```
-- Left Identity
(return a) >>= f = f a

-- Right Identity
m >>= return = m

-- Associativity
(m >>= f) >>= g = m >>= (\x -> f x >>= g)
```

Returning to the Bank Example

Now that we have this concept of monads in Haskell, let's put them to use. Consider a writer monad. Our writer type will be a tuple that holds an object and a string (for logging purposes). Now, we can define a functor and monad for this type.

```
-- Create a new type
data Writer a = Writer (a, String)

-- We can create a Functor that defines the morphisms
instance Functor Writer where
    fmap :: (a -> b) -> Writer a -> Writer b
    fmap f (Writer (a, s)) = Writer (f a, s)

-- We can now define the Monad
instance Monad Writer where
    return x = Writer (x, "")
    m (>>=) f = Z where
      Writer (x, s1) = m
      Writer (y, s2) = f m
      Z = Writer(y, s1 ++ s2)
```

So, what are we doing with `(>>=)`? We're creating composition between our functions. Now consider what our bank program would look like.

```
-- Now we can define a new login
login :: String -> Writer (Int, String)
login x = Writer (old_login x, "login")

-- and a new withdraw
withdraw :: Int -> Writer (Int, String)
withdraw x = Writer (old_withdraw x, "withdraw")

-- to compose we now use (>>=) bind
login x >>= withdraw

-- the steps of bind expanded would be
-- 1. login x = Writer (old_login x, "login")
-- output becomes input
-- 2. withdraw (old_login x) = Writer (old_withdraw (old_login x), "withdraw")
-- then combine!
-- 3. Writer(old_withdraw (old_login x), "login" ++ "withdraw")
```

So, what are the advantages of this approach? For one, we don't need to know how to add things to logs, that's abstracted away by the bind operator. We could easily define a new bind operator that does something slightly different and not have to change every single function that employs the Writer monad. We also keep privacy since we don't see the end result. For each function we only worry about what we want to output to the writer and we don't see anything else.

Let's take a look at some other examples.

Example: Maybe Monad, Haskell Style

We've already seen the **Maybe** typeclass as a functor. Let's take a look at it as a functor.

```
-- As a refresher, here is the type declaration
data Maybe a = Just a | Nothing

-- Functor definition
instance Functor Maybe where
    fmap :: (a -> b) -> Maybe a -> Maybe b
    fmap _ Nothing = Nothing
    fmap f Just x = Just (f x)

-- Monad definition
```

```
instance Monad Maybe a where
    return :: a -> Maybe a
    return x = Just x

    (>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
    m >>= f = case m of
                Nothing -> Nothing
                Just x -> f x
```

Just like in the category example, the Maybe type is just an extra object **Nothing** that's been added to our existing type. **Nothing** signifies incomplete or invalid computation, similar to a Null Pointer.

Using the maybe we can chain together partial computations with the bind operator. Consider for example, a program that finds an ancestor from a family tree.

```
-- We have functions that get the mother and father (which may not exist)
mother :: Person -> Maybe Person
father :: Person -> Maybe Person

-- Now we want to get the maternal grandfather
maternal_grandfather :: Person -> Maybe Person
maternal_grandfather p = (mother p) >>= father

-- What about your father's mother's mother's father?
some_guy :: Person -> Maybe Person
some_guy = father p >>= (\x -> mother x >>= (\y -> mother y >>= (\z -> father x)))
-- above, we're just chaining the bind operator.

-- Haskell gives us special notation for this
some_guy = do x <- father p
              y <- mother x
              z <- mother y
              father z
-- uh oh we've just reinvented imperative programming! :)
```

Without this concept of monads, it would be much more difficult to find some arbitrary ancestor. Notice that if any of the functions return **Nothing** the computation will just evaluate to **Nothing**, no need to handle this case explicitly. This is of course a toy example, but you can see how this idea of composability is very powerful and scales quite well.

List Monad, Haskell Style

Lists are also monads in Haskell. Let's take a look in Haskell notation.

```
-- Remember that the List as a Functor
instance Functor [a] where
    fmap :: (a -> b) -> [a] -> [b]
    fmap = map

-- Definition of the List Monad
instance Monad [a] where
    return :: a -> [a]
    return x = [x]

    -- the concat operator simply joins all the lists that are in the list together
    join :: [[a]] -> [a]
    join = concat

    (>>=) :: [a] -> (a -> [b]) -> [b]
    m >>= f = concat (fmap f m)
```

What's fun here is that we can see that **join** and `fmap` really do compose into bind.

Instead of looking at examples, let's check that **List** is indeed a monad by the monad laws.

Left Identity

```
return a >>= f = [a] >>= f
               = concat (fmap f [a])
               = concat [f a]          concatenating a singleton list
               = f a
```

Right Identity

```
m >>= return = concat (fmap return m)
             = concat [return m_1, return m_2, ... , return m_n]
             = concat [[m_1], [m_2], ... , [m_n]
             = m
```

Associativity

```
(m >>= f) >>= g) = (concat (fmap f m) >>= g)
                 = concat (fmap g (concat (fmap f m)))
                 = let m = x in concat (fmap g (concat (fmap f x)))
                 = let m = x in x >>= f >>= g
                 = m >>= (x-> f x >>= g x)
```

So, **List** is a monad.

4.2.3 Summary of Monads

At the end of the day, it is doubtful that every single programmer understands and fully internalizes what exactly monads are and how they are actualized in code, even those fluent in Haskell. Even if it is not absolutely necessary to understand the category theory behind implemented monads, it is definitely essential to how Haskell functions function. We see this all the way from hassle-free error-checking up to simplified composition of what could otherwise be very complicated algorithms.

And while there isn't one way monads function (for example, a Maybe monad behaves very differently from a List monad), its versatility in its implementation does not take away from the consistently inherent benefits we get from it such as building exceptions into code, having more flexibility with functions (in terms of whenever one might need to adapt new code from old ones), non-determinism, avoiding global variables, and much more.

5. Conclusion

At first glance, it seems quite odd that category theory, an abstract branch of mathematics is so closely tied with any programming language. Category theory is all about abstracting patterns and structures that can be used to generalize concepts. Computer science is all about computation and producing results and systems. However, as we've discovered, the two concepts are actually quite closely tied. In fact, if we dig deeper we can see that the nature of computer science is in fact abstraction. Computers processing comes from computer chips, which consist of billions of tiny transistors (electrically controlled switches), that produce basic logic. From thos basic building blocks, we abstract further and further up the stack. It is the programmers jobs to handle these abstractions, building more and more complex structures on top of these basic building blocks. Haskell, as a high-level functional programming language is an amazing environment for these categorical concepts.

First, we saw how the concept of functors can be translated naturally into operations in Haskell. We saw examples of **Maybe** and **List**. Then, we explored the concept of monoids in Haskell. Next, we saw how monoids are generalized to categories. In fact, monads are monoids in the category of of endofunctors. Finally, we saw the application of monads in Haskell and their general usefulness in computer science. Monads are a powerful tool because they allow us to generalize the idea of composition, the essence of functional programming. Thanks for joining us on this journey!

A. First Appendix

Division of tasks:

- Sinay: Entire Chapter 3 on Monoids including categorical definition, definition of monoids in haskell, and examples in haskell), and entire subchapter 5.1 on monads in category theory (including definition, adjunction example, maybe monad, list monad, kleisli category definition, kleisli on maybe, kleisli on list), and finally summary of monads
- Greg: Introduction Chapter 1, Functor Chapter 2, Monads in Haskell Section 5.2

Bibliography

[1] Samuel Grahn. Monads in haskell and category theory. 2019. http://uu.diva-portal.org/smash/get/diva2:1369286/FULLTEXT01.pdf.

[2] Miran Lipovaca. *Learn You a Haskell for Great Good! A Beginner's Guide*. No Starch Press, USA, 1st edition, 2011.

[3] Bartosz Milewski. *Category Theory For Programmers*. 2019. https://github.com/hmemcpy/milewski-ctfp-pdf.

[4] Emily Riehl. *Category Theory In Context*. 2014.

[5] Wikibooks. Haskell/category theory — wikibooks, the free textbook project, 2019. [Online; accessed 3-December-2021].